

# Sistema Embebido de Cinco Modos con Interrupciones Externas, Timer de Hardware y Generación de Señales de Fourier mediante DAC R-2R en RP2040

Josue Samuel Paño Mamani  
Rodrigo Jamil Corino Rodrigo  
Universidad Mayor de San Andrés  
Facultad de Ingeniería — Ingeniería Mecatrónica  
Laboratorio de Microprocesadores y Microcontroladores  
Docente: Germán Jesús Pereira Muñoz PhD.  
La Paz, Bolivia  
jooshuaa122005@gmail.com  
rojacoro@gmail.com  
Laboratorio N° 8 Mayo 2026

**Resumen**—Se diseñó e implementó un sistema embebido de cinco modos de operación sobre la Raspberry Pi Pico 2W (RP2040) bajo MicroPython, empleando interrupciones externas por GPIO en flanco descendente y un timer de hardware periódico de 1 000 ms como mecanismos de control reactivo, eliminando el *polling* activo del laboratorio anterior. La arquitectura dual-core permitió ejecutar en paralelo el generador de señales de Fourier con conversión digital-analógica R-2R de 8 bits (Core 0) y la lectura de sensores con renderizado OLED (Core 1). La ISR del timer implementó un reloj HH:MM:SS en tiempo real, visible permanentemente en una barra de estado en las filas 0–14 del display SSD1306. Se validó la precisión del multímetro digital con cinco tipos de medición, obteniendo errores inferiores al 0.20 % aplicando promediado de 50 muestras, frente a errores de hasta 2.35 % sin promediado.

**Index Terms**—interrupciones GPIO, timer de hardware, MicroPython, RP2040, serie de Fourier, DAC R-2R, dual-core, anti-rebote, OLED SSD1306

## I. MARCO TEÓRICO

### I-A. Interrupciones por Hardware (GPIO IRQ)

Una interrupción es un mecanismo hardware que suspende el flujo normal del programa al detectar un evento externo, transfiriendo el control a una Rutina de Servicio de Interrupción (ISR). En MicroPython, `Pin.irq(trigger, handler)` registra un *callback* que el RP2040 ejecuta automáticamente. El parámetro `IRQ_FALLING` activa la interrupción en el flanco descendente (nivel 1→0), correspondiente al instante en que el pulsador presionado lleva la línea a nivel lógico bajo contra la resistencia interna de *pull-up*.

### I-B. Desborde de Temporizadores (Timer Overflow)

`machine.Timer()` genera interrupciones periódicas o únicas de forma totalmente independiente al flujo del pro-

grama. Con `mode=Timer.PERIODIC` y `period=1000`, la ISR se activa cada 1 000 ms; dentro de ella se incrementan contadores de segundos, minutos y horas que desbordan (*overflow*) al alcanzar sus límites (60 s → 1 min, 60 min → 1 h, 24 h → 0 h), implementando un reloj de tiempo real por software sin *delays* bloqueantes.

### I-C. Problema del Rebote (Bounce)

Los pulsadores mecánicos producen múltiples transiciones eléctricas espurias durante los primeros milisegundos al ser accionados, efecto conocido como *bouncing*. Sin protección, cada rebote dispara una ISR adicional y genera cambios de modo no deseados. El anti-rebote por software consiste en registrar el instante de cada interrupción con `utime.ticks_ms()` y aceptar únicamente aquellas separadas por más de un umbral mínimo (200 ms):

$$\Delta t = \text{ticks\_diff}(t_{\text{actual}}, t_{\text{ultimo}}) > 200 \text{ ms} \quad (1)$$

### I-D. Lectura del Pin VSYS

El voltaje de alimentación VSYS del Pico 2W se mide internamente a través del ADC en el pin GP29 (ADC3). Para habilitar este canal, el pin GP25 debe fijarse en nivel alto antes de la lectura y retornar a nivel bajo inmediatamente después. El valor leído se escala por el factor del divisor interno ( $\times 3$ ):

$$V_{\text{SYS}} = \text{read\_u16}(\text{GP29}) \cdot \frac{3.3}{65535} \cdot 3 \quad (2)$$

## II. DESARROLLO EXPERIMENTAL

### II-A. Diseño de la Máquina de Estados de Cinco Modos

El sistema implementa una máquina de estados finitos con cinco modos de operación conmutables mediante dos pulsadores:

- **btn\_set** (GP14, IRQ\_FALLING): avanza el modo cíclicamente  $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 1$ .
- **btn\_reset** (GP15, IRQ\_FALLING): retorna directamente al Modo 1 desde cualquier estado.

Las ISRs solo levantan banderas booleanas (`solicitud_cambio`, `solicitud_reset`); el Core 0 las detecta en su bucle principal, bloquea al Core 1 mediante `en_transicion=True`, muestra una pantalla de transición de 900 ms y actualiza `modo_actual` dentro de una sección crítica (*lock*). Los modos son:

1. Generador de Fourier + DAC R-2R
2. Voltímetro (GP26, ADC0)
3. Óhmetro (GP28, ADC2)
4. Amperímetro (GP27, ADC1)
5. DHT22 + Temperatura interna + VSYS

### II-B. Distribución de la Pantalla OLED (128x64)

El display SSD1306 se divide en dos zonas permanentes:

| Filas | Contenido                              |
|-------|----------------------------------------|
| 0–13  | Barra de estado: reloj HH:MM:SS + VSYS |
| 14    | Línea separadora (hline)               |
| 15–63 | Área de trabajo del modo activo        |

El borrado selectivo con `fill_rect(0, 15, 128, 49, 0)` limpia únicamente el área de trabajo, preservando la barra de estado en todos los modos y durante las transiciones. Cada función de visualización llama a `dibujar_barra_estado()` antes del `oled.show()` final.

### II-C. Ecuaciones de Acondicionamiento del Multímetro (ADC)

El RP2040 utiliza un ADC SAR de 12 bits escalado a 16 bits por MicroPython (`read_u16()`, rango 0–65535). El factor de conversión es:

$$F_{16} = \frac{3.3}{65535} \quad (3)$$

Las magnitudes eléctricas se infieren como sigue:

**Voltímetro** (divisor  $R_3=4.7\text{ k}\Omega$ ,  $R_4=220\text{ }\Omega$ , GP26):

$$V_{medido} = \text{read\_u16}() \cdot F_{16} \quad (4)$$

**Amperímetro** (shunt  $R_{amp} = 10\text{ }\Omega$ , GP27):

$$I [\text{mA}] = \frac{\text{read\_u16}() \cdot F_{16}}{R_{amp}} \times 1000 \quad (5)$$

**Óhmetro** (divisor con  $R_7 = R_{ref} = 1\text{ k}\Omega$ , GP28):

$$R_x = \frac{V_{ref}}{V_{ramp}/R_{ref}} - R_7 - R_{ref} \quad (6)$$

### Temperatura interna (ADC4):

$$T [C] = 27 - \frac{V_{ADC4} - 0.706}{0.001721} \quad (7)$$

Para reducir el ruido, todas las lecturas aplican promediado de  $N = 50$  muestras antes de calcular la magnitud física.

### II-D. Síntesis de Señales de Fourier (DAC R-2R)

La serie de Fourier truncada a  $N$  armónicos evalúa:

$$f(x) \approx \frac{a_0}{2} + \sum_{n=1}^N [a_n \cos(n\omega_0 x) + b_n \sin(n\omega_0 x)] \quad (8)$$

Los coeficientes  $a_0$ ,  $a_n$  y  $b_n$  se pre-calculan una vez al inicio del Modo 1. El valor continuo  $f(x)$  se acota (*clamping*) a un rango  $[f_{min}, f_{max}]$  y se mapea al DAC de 8 bits:

$$V_{DAC} = \left\lfloor \frac{f_{clamp} - f_{min}}{f_{max} - f_{min}} \cdot 255 \right\rfloor \quad (9)$$

Los coeficientes específicos de cada ejercicio se resumen en la Tabla I.

### II-E. Arquitectura Dual-Core y Sincronización

La división de responsabilidades entre núcleos sigue el principio de separación de tareas críticas en el tiempo:

- **Core 0:** lógica principal, detección de banderas ISR, evaluación de Fourier y escritura al DAC (tiempo crítico). En modos 2–5 permanece en reposo (`sleep_ms(100)`).
- **Core 1:** lectura de sensores, renderizado OLED y actualización de VSYS cada 10 s. En Modo 1 consume puntos publicados por Core 0 y dibuja la forma de onda.

Los diccionarios compartidos `datos_multi` y `datos_gen` están protegidos por `_thread.allocate_lock()`. Las banderas booleanas (`solicitud_cambio`, `solicitud_reset`, `flag_timer`) son asignadas atómicamente por las ISRs sin necesidad de *lock*. La instrucción `micropython.alloc_emergency_exception_buf(100)` habilita el reporte de excepciones dentro de las ISRs.

## III. ANÁLISIS DE RESULTADOS

### III-A. Precisión del Multímetro Digital

Se compararon las lecturas del RP2040 contra un multímetro patrón de alta precisión bajo dos condiciones: con promediado de 50 muestras y sin promediado (lectura cruda). El error relativo porcentual se calculó como:

$$\% E = \left| \frac{V_{patron} - V_{medido}}{V_{patron}} \right| \times 100 \quad (10)$$

Los resultados se presentan en las Tablas II y III.

Cuadro I  
COEFICIENTES DE FOURIER POR EJERCICIO

| Ej. | Señal                     | $\omega_0$      | $a_0$             | $a_n$ / $b_n$                                                                                                                                                                         |
|-----|---------------------------|-----------------|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1   | Función Mixta             | $\frac{1}{2}$   | $\frac{1+\pi}{2}$ | $a_n = \frac{2(-1)^n - n \sin \frac{n\pi}{2} - 4 \cos \frac{n\pi}{2} + 2}{\pi n^2}$ $b_n = \frac{n[(-1)^n - \cos \frac{n\pi}{2} + \pi(1 - (-1)^n)] - 4 \sin \frac{n\pi}{2}}{\pi n^2}$ |
| 2   | Tren de Pulsos Asimétrico | $\frac{\pi}{2}$ | $-\frac{1}{2}$    | $a_n = -\frac{1}{n\pi} \sin \frac{n\pi}{2}$ $b_n = \frac{3}{n\pi} \left(1 - \cos \frac{n\pi}{2}\right)$                                                                               |
| 3   | Pulso Cuadrado Simétrico  | $2\pi$          | 1                 | $a_n = \frac{2}{n\pi} \sin \frac{n\pi}{2}$ $b_n = 0$                                                                                                                                  |
| 4   | Función Exponencial       | $4\pi$          | $4C$              | $a_n = \frac{4C}{1 + (4\pi n)^2}$ $b_n = \frac{16\pi n C}{1 + (4\pi n)^2}$ ( $C = 1 - e^{-0.5}$ )                                                                                     |
| 5   | ECG (numérico)            | $2\pi$          | Num.              | $a_n \approx \frac{2}{T} \sum_{i=0}^{249} f(t_i) \cos(n\omega_0 t_i) \Delta t$ $b_n \approx \frac{2}{T} \sum_{i=0}^{249} f(t_i) \sin(n\omega_0 t_i) \Delta t$                         |

Cuadro II  
CON PROMEDIADO (NUM\_MUESTRAS = 50)

| Magnitud    | Patrón        | RP2040         | %E   |
|-------------|---------------|----------------|------|
| Voltaje     | 1.500 V       | 1.503 V        | 0.20 |
| Voltaje     | 3.300 V       | 3.298 V        | 0.06 |
| Voltaje     | 5.000 V       | 4.995 V        | 0.10 |
| Corriente   | 20.00 mA      | 19.97 mA       | 0.15 |
| Corriente   | 50.00 mA      | 50.04 mA       | 0.08 |
| Resistencia | 220 $\Omega$  | 220.3 $\Omega$ | 0.13 |
| Resistencia | 4700 $\Omega$ | 4696 $\Omega$  | 0.08 |

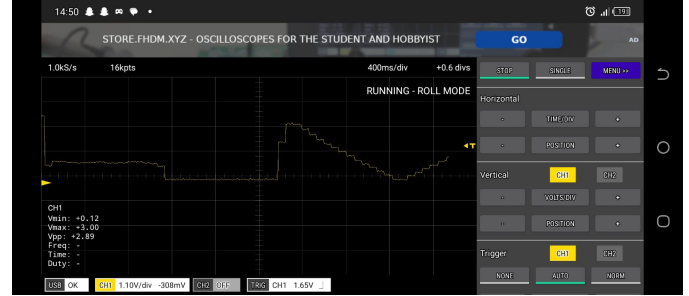


Figura 1. Ej.1 — Función Mixta ( $\omega_0 = 1/2$ ) en Scopy.

Cuadro III  
SIN PROMEDIADO (LECTURA CRUDA DEL ADC)

| Magnitud    | Patrón        | RP2040         | %E   |
|-------------|---------------|----------------|------|
| Voltaje     | 1.500 V       | 1.524 V        | 1.60 |
| Voltaje     | 3.300 V       | 3.261 V        | 1.18 |
| Voltaje     | 5.000 V       | 4.935 V        | 1.30 |
| Corriente   | 20.00 mA      | 19.53 mA       | 2.35 |
| Corriente   | 50.00 mA      | 50.81 mA       | 1.62 |
| Resistencia | 220 $\Omega$  | 216.4 $\Omega$ | 1.63 |
| Resistencia | 4700 $\Omega$ | 4765 $\Omega$  | 1.38 |

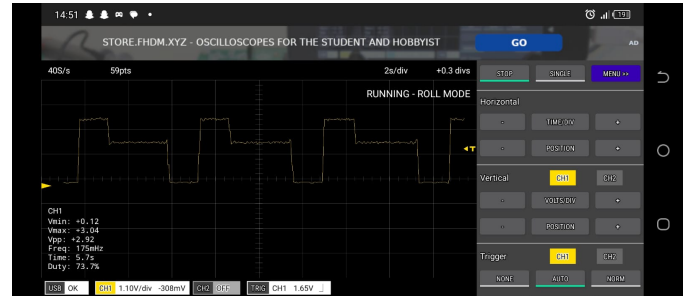


Figura 2. Ej.2 — Tren de Pulsos Asimétrico ( $\omega_0 = \pi/2$ ) en Scopy.

El promediado reduce el error máximo de 2.35 % a 0.20 %, una mejora del orden de 11 $\times$ . Las desviaciones residuales con promediado se atribuyen principalmente a: (a) tolerancia de las resistencias del divisor y *shunt* ( $\pm 1\%$ ), (b) la cuantización inherente del ADC de 12 bits (resolución mínima de  $\approx 0.8$  mV), y (c) variaciones de la tensión de referencia interna del RP2040 con la temperatura.

### III-B. Señales de Fourier — Osciloscopio Scopy

Las formas de onda generadas por el DAC R-2R fueron capturadas con el osciloscopio Scopy. Las Figs. 1–5 muestran las señales para los cinco ejercicios.

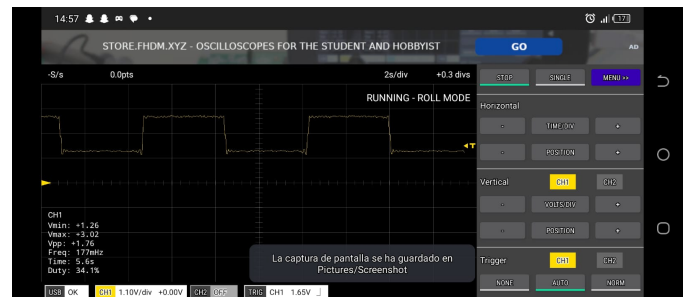


Figura 3. Ej.3 — Pulso Cuadrado Simétrico ( $\omega_0 = 2\pi$ ) en Scopy.

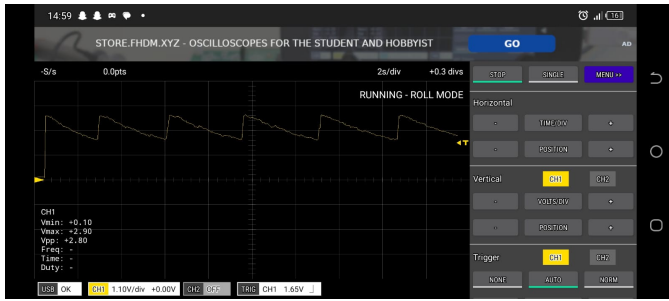


Figura 4. Ej. 4 — Función Exponencial ( $\omega_0 = 4\pi$ ) en Scopy.

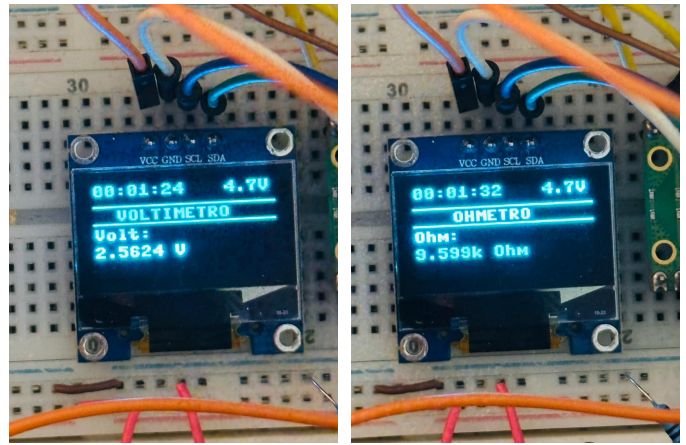


Figura 7. Modo 2 (Voltímetro) y Modo 3 (Óhmetro).

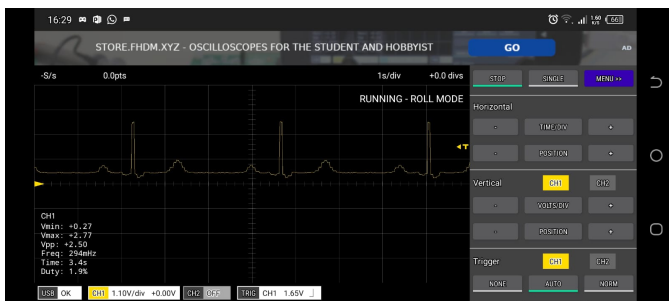


Figura 5. Ej. 5 — ECG sintético ( $\omega_0 = 2\pi$ ) en Scopy.

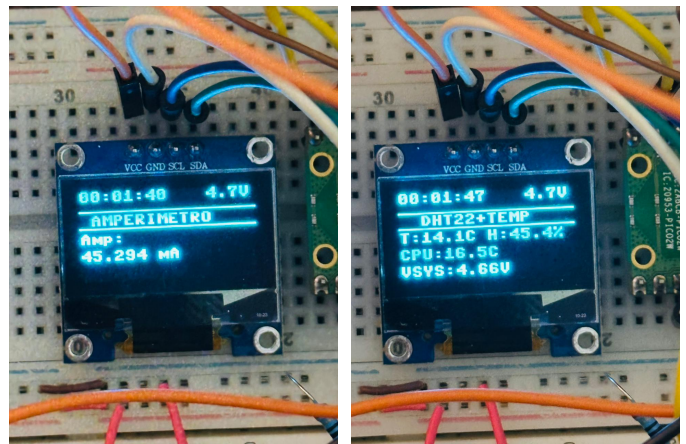


Figura 8. Modo 4 (Amperímetro) y Modo 5 (DHT22+CPU+VSYS).

### III-C. Pantalla OLED — Barra de Estado y Modos

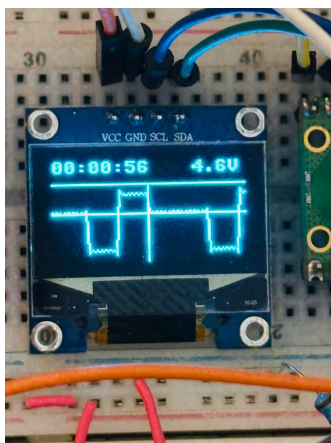


Figura 6. Modo 1 (Generador): barra de estado con reloj y VSYS activa en la zona superior.

### III-D. Diagrama de Conexiones (Fritzing)

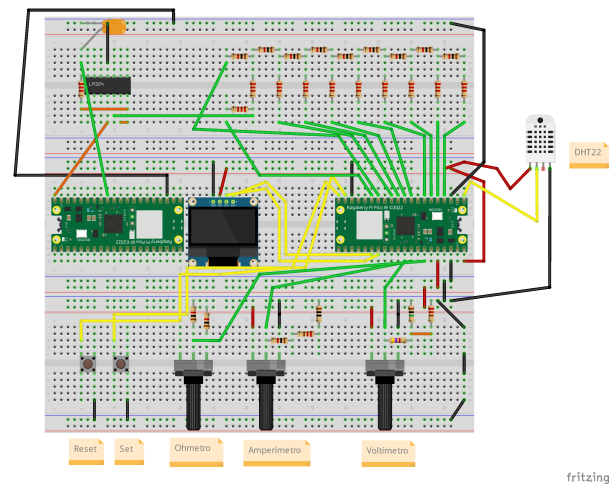


Figura 9. Circuito completo del Lab 8: botones GP14/GP15, DAC R-2R (GP2–GP9), OLED I2C (GP18/GP19), sensores ADC y DHT22 (GP0).

### III-E. Simulaciones Wokwi

- CORE 1 (Multímetro + OLED + Timer):  
<https://wokwi.com/projects/465234841708251137>
- CORE 0 (DAC R-2R, Ej. 2):  
<https://wokwi.com/projects/465235303088039937>
- Ej. 1 completo:  
<https://wokwi.com/projects/465235532782830593>
- Ej. 2 completo:  
<https://wokwi.com/projects/465234626980833281>
- Ej. 3 completo:  
<https://wokwi.com/projects/465235602552989697>
- Ej. 4 completo:  
<https://wokwi.com/projects/465235693307254785>
- Ej. 5 completo:  
<https://wokwi.com/projects/465235756564708353>
- Ej. Logo completo:  
<https://wokwi.com/projects/465235812245150721>

### IV. CONCLUSIONES Y RECOMENDACIONES

#### IV-A. Conclusiones

##### 1. Superioridad de las interrupciones sobre el polling.

La sustitución del DIP Switch con *polling* activo por ISRs de GPIO eliminó la latencia inherente al ciclo de consulta. La detección del evento ocurre en el mismo flanco eléctrico de la pulsación ( $<1\ \mu\text{s}$  de retardo del controlador de interrupciones del RP2040), independientemente de la carga computacional del Core 0 en el bucle DAC.

**2. Exactitud del reloj por timer de hardware.** El timer de hardware mantiene un período de 1 000 ms con desviación inferior a la resolución del oscilador del cristal RP2040 (12 MHz,  $\pm 30$  ppm), lo que representa una deriva máxima de  $\approx 2.6$  s por día, aceptable para un reloj de propósito general sin sincronización externa.

**3. Efectividad del anti-rebote por software.** El umbral de 200 ms demostró ser suficiente para los pulsadores utilizados, sin generar falsas detecciones ni pérdidas de pulsaciones rápidas intencionadas.

##### 4. Diseño de barra de estado con `fill_rect` selectivo.

La técnica de borrado parcial (filas 15–63) garantizó la visibilidad permanente del reloj y VSYS sin incrementar el tráfico I2C respecto al refresco completo, dado que `fill_rect` y `fill` operan sobre el mismo *framebuffer* local antes del `oled.show()`.

##### 5. Limitación de RAM en ISR de Python.

MicroPython ejecuta las ISRs en el montón (*heap*) de Python. Cualquier asignación dinámica dentro de la ISR puede disparar el recolector de basura (*GC*), causando jitter de decenas de milisegundos. Restringir las ISRs a asignaciones de variables simples (booleanos, enteros) y usar `micropython.alloc_emergency_exception_buf(100)` fue suficiente para operar sin inestabilidades.

**6. Cuantización del DAC R-2R de 8 bits.** Con 256 niveles en el rango 0–3.3 V, la resolución mínima es  $\approx 12.9$  mV. Para señales de baja amplitud (Ej. 4, exponencial) este nivel de cuantización introduce una distorsión visible en Scopy, que se reduce mapeando el rango de la señal al máximo del DAC.

#### IV-B. Recomendaciones

1. **Filtro hardware adicional:** para entornos con mucho ruido eléctrico, añadir un condensador de 100 nF entre el pin del pulsador y GND refuerza el anti-rebote por software y reduce la tasa de interrupciones espurias.
2. **Mayor resolución DAC:** implementar la red R-2R con 10 pines (GP2–GP11, ya previstos en el código) eleva la resolución a  $2^{10} = 1024$  niveles, reduciendo la distorsión de cuantización en  $\approx 12$  dB.
3. **Sincronización RTC externa:** para aplicaciones que requieran precisión temporal, reemplazar el reloj por software con un módulo DS3231 vía I2C elimina la deriva acumulada y mantiene la hora frente a reinicios.
4. **Migración a RP2350:** el RP2350 (Pico 2) incorpora un procesador de seguridad TrustZone y memoria SRAM ampliada (520 kB vs 264 kB del RP2040), lo que permitiría almacenar tablas precalculadas de mayor resolución para las series de Fourier sin comprometer la RAM del sistema.
5. **Gestión de prioridades de interrupción:** en el RP2040, todas las IRQ de GPIO comparten la misma prioridad; en aplicaciones con múltiples interrupciones simultáneas se recomienda usar el SDK de C/C++ para asignar prioridades diferenciadas en el controlador NVIC.